

Keepriva UI Restructure, Item History, and Parallel Test Execution Plan

1. Document purpose

This document defines the sequential implementation plan for the next Keepriva Android development cycle. It covers:

- A KeePass2Android-inspired, independently implemented vault browsing experience.
- A separate item-search results view.
- Copy-to-clipboard actions for every displayed item value.
- Explicit movement of an item between categories.
- Item creation and modification timestamps.
- Encrypted item version history, version inspection, restoration, and removal.
- Backup and restore compatibility for version history.
- Production tests and instrumentation tests delivered with every phase.
- Refactoring the GitHub Actions workflow so emulator tests run in isolated logical batches in parallel.
- A behavior-preserving modularization of `MainActivity` before feature-heavy UI phases begin.

This is an implementation roadmap, not an authorization to change the current green branch. Each phase must be implemented, reviewed, tested, and merged independently.

2. Confirmed baseline

Baseline item	Current value
Repository	subinks/Keepriva_Android
Branch	ui_eh_ph02_screen_routing
Phase 2 code baseline commit	2212df03ad713e5693bfd8d6b46b7d5ae926b363
Phase 2 green workflow run	36100647077
Original Phase 0 workflow duration	Approximately 18 minutes 22 seconds
Android instrumentation tests	95 methods across 9 test classes
Normal blocking tests	94
Biometric CI diagnostic	1 conditional/non-blocking test
JVM router/model tests	9
Database version	3
Encrypted backup format version	1
Application architecture	One 3,684-line <code>MainActivity</code> , Phase 2 router/root infrastructure, mixed programmatic and reusable XML views

After the corrected planning documents pass the unchanged Phase 2 workflow, create a distinct Phase 2 baseline tag from the latest green head of `ui_eh_ph02_screen_routing`:

```
ui-enhancement-phase2-green-2026-09-25
```

Do not move or reuse the existing Phase 0 tag `ui-enhancement-green-2026-09-24`.

Phase 2A must begin from that latest green Phase 2 branch head, including these planning documents, on a new branch:

```
ui_eh_ph02a_mainactivity_modularization
```

Do not add the Phase 3 browser redesign directly to the monolithic activity. Complete and prove the behavior-preserving Phase 2A extraction first.

3. Scope and non-goals

3.1 In scope

- 1 Restructure the vault home screen into category navigation and compact item lists.
- 2 Introduce dedicated full-screen views for search, item details, history, and historical-version details.
- 3 Preserve Keepiva's green/white identity and all current security behavior.
- 4 Add field-level secure clipboard actions.
- 5 Add explicit item movement between categories.
- 6 Display item creation and modification timestamps.
- 7 Add encrypted historical snapshots for item changes.
- 8 Extend encrypted backup and restore to include history.
- 9 Update existing tests and add new tests in the same phase as each production change.
- 10 Parallelize emulator verification by logical feature group.
- 11 Modularize `MainActivity` into lifecycle/session orchestration, feature controllers, repositories, view-state builders, and reusable UI factories before later UI work.

3.2 Out of scope for this cycle

- Dropbox, cloud, WebDAV, or network synchronization.
 - Adding the `INTERNET` permission.
 - Copying KeePass2Android source code, icons, assets, or layouts.
 - Changing Keepiva's encryption hierarchy or master-password derivation unless a separately reviewed security need is found.
 - Autofill or a custom Keepiva keyboard.
 - Multi-device conflict resolution.
 - Multiple vault files.
-

4. Architectural decisions to make once and retain

These decisions prevent repeated design changes during implementation.

4.1 Retain a single activity as the lifecycle/session boundary

`MainActivity` remains the Android lifecycle boundary and composition root. Its activity-scoped `VaultSessionCoordinator` is the sole in-memory owner of the active `SecretKey`; `MainActivity` must not retain a second key field. Phase 2A reduces the activity to coordination of the session lifecycle, router, root renderer, lock lifecycle, Activity results, and feature controllers.

Feature controllers receive narrow callback interfaces. They must not retain a `SecretKey`, decrypted `VaultItem`, `Activity`, dialog, or view beyond the lifetime of the rendered screen. Database and cryptographic operations are invoked through activity-owned/session-scoped gateways that accept the key only for the duration of a call.

Do not pass the vault key through:

- `Intent` extras.
- `Bundle` values.
- Saved-instance state.
- Files or shared preferences.
- Static global variables.

4.2 Introduce an internal screen-navigation layer

Phase 2 introduced the router. Phase 2A extracts event handling and rendering from the large activity while retaining one Android activity:

```
MainActivity
  MainActivityCompositionRoot
    VaultScreenRouter
    VaultSessionCoordinator
    VaultRootRenderer
    ActivityResultCoordinator
    SetupController
    UnlockController
    SecuritySettingsController
    LegacyVaultBrowserController
    CategoryManagementController
    ItemDialogController
    DataTransferController
```

The `LegacyVaultBrowserController` and `ItemDialogController` are temporary behavior-preserving extractions. Phase 3 replaces the legacy browser controller; Phase 5 replaces item-detail dialogs with the final full-screen controller. This prevents Phase 2A from silently becoming a UI redesign.

`MainActivity` remains responsible for sensitive state and supplies callbacks to controllers. The screen router maintains a small in-memory navigation stack and clears it immediately when the vault locks.

This approach avoids a high-risk, all-at-once migration to multiple activities or a new application framework. Fragments can be considered later, after the UI and security behavior are stable.

4.3 Use reusable XML layouts and list row components

Move new screens and reusable rows to XML resources. Use a `RecyclerView` with stable item IDs for:

- Category rows.
- Item rows.
- Search results.
- Version-history rows.

Do not rebuild an entire long `LinearLayout` tree for every search keystroke.

4.4 Preserve semantic accessibility identifiers

Every screen and action requires a stable semantic identifier/content description. Tests should prefer these identifiers over visible wording.

Examples:

```
Screen vault browser
Screen search results
Screen item details
Screen item history
Open category Work
Open item GitHub Admin
Copy Username / Email
Move item
Restore item version
Remove item version
```

4.5 Use full-screen views for primary workflows

Dialogs should remain only for short decisions, confirmation, reauthentication, and small option selectors. Search, item details, editing, and history should be full-screen views.

This reduces cramped layouts and avoids the dialog-root/window-focus failures previously encountered by Espresso.

5. Target screen flow

```

flowchart TD
    A[Vault Browser] --> B[Category Contents]
    A --> C[Search Results]
    B --> D[Item Details]
    C --> D
    D --> E[Edit Item]
    D --> F[Move Item]
    D --> G[Item History]
    G --> H[Version Details]
    H -->|Restore| D
    H -->|Remove version| G

```

The Android Back action must return to the previous screen and restore the prior category, query, and scroll position. Locking the vault must discard the complete screen stack and show the unlock screen.

6. Phase overview

Phase	Primary outcome	Database change	Backup change	Main test impact
0	Freeze baseline and record measurements	No	No	Preserve 89-test baseline
1	Parallel CI/emulator foundation	No	No	Split existing tests into isolated batches
2	Screen router and reusable UI foundation	No	No	Navigation and lifecycle tests
2A	Behavior-preserving MainActivity modularization	No	No	Controller contracts, wiring, lifecycle, and full 95-test regression
3	Category-first vault browser redesign	No	No	Update home/category/smoke tests
4	Separate search-results screen	No	No	New search-results suite
5	Full-screen details, timestamps, and copy actions	No	No	New clipboard/details suite
6	Explicit move-item workflow	No	No	New move-item suite
7	Encrypted version-history storage	v3 to v4	No	Migration and history repository tests
8	History and version-detail UI	Already v4	No	New history UI suite
9	Version-aware backup and restore	No	v1 to v2	Backup compatibility and round-trip tests
10	Integration, accessibility, visual verification, release gate	No	No	Full regression and screenshots

Each phase is a merge boundary. Do not start the next phase until the current phase's exit criteria are green. Phase 2A is a mandatory dependency for every phase from Phase 3 onward.

7. Detailed sequential implementation plan

Phase 0 - Freeze and document the green baseline

Objective

Create a recoverable starting point and capture the current workflow/test characteristics before any structural change.

Production tasks

- 1 Tag commit `6851c467896abc3d8d375cfb9f237379adce5708`.
- 2 Create `ui_restructure_v2` from that tag/commit.
- 3 Record the current APK version, database version, and backup format version.
- 4 Record the latest successful workflow duration and each job/step duration.
- 5 Save the current setup, home, and unlock screenshots as baseline artifacts.
- 6 Confirm the manifest contains no `INTERNET` permission.

Test tasks

- 1 Run the complete existing suite unchanged.
- 2 Confirm 88 normal tests pass.
- 3 Confirm the biometric CI test remains conditional/non-blocking.
- 4 Archive the test output and screenshot artifacts.

Exit criteria

- The branch is created from the known green commit.
- The baseline tag exists remotely.
- The unchanged workflow is green.
- Baseline artifacts are retained for UI comparison.

Rollback point

Return to the protected baseline tag.

Phase 1 - Build once and execute emulator tests in parallel batches

Objective

Reduce feedback time before the test suite grows, while preserving failure isolation and complete coverage.

Workflow restructuring

Replace the current single `build-and-verify` job with these logical jobs:

```
build-apks
|
+--> instrumentation matrix (parallel)
|
+--> visual-verification (parallel)
|
+--> verification-gate
```

build-apks job responsibilities

- 1 Checkout source.
- 2 Set up JDK 17 and Gradle 9.1.0.
- 3 Install Android SDK 36 build components.
- 4 Apply the existing disposable debug screenshot-protection bypass.
- 5 Verify there is no `INTERNET` permission.
- 6 Run:

```
:app:lintDebug
:app:testDebugUnitTest
:app:assembleDebug
```

```
:app:assembleDebugAndroidTest
```

- 1 Upload one immutable APK bundle containing:
- 2 `app-debug.apk`.
- 3 The instrumentation APK.
- 4 APK checksums.
- 5 Upload lint and JVM unit-test reports.

No emulator should be started in this job.

Initial instrumentation matrix for the current 89 tests

Batch ID	Test classes	Current test methods	Purpose
auth-lifecycle-smoke	KeeprivaAuthHomeTest, KeeprivaLifecycleRobustnessTest, KeeprivaUiSmokeTest	26	Setup, unlock, lock, recreation, basic reachability
categories	KeeprivaCategoryPreferencesTest, KeeprivaCategoryDeletionTest	24	Category hierarchy, preferences, add/edit/delete behavior
item-core	KeeprivaItemCrudTest	14	Item creation, details, password display, search, delete/undo
data-transfer	KeeprivaDataTransferTest	12	Import, export, backup document intents and dialogs
security	KeeprivaSecuritySettingsTest	12	Reauthentication, auto-lock, clipboard, password and biometric settings

The remaining `KeeprivaBiometricCiTest` stays in the separate visual/biometric diagnostic job because emulator biometric enrollment is environment-dependent.

Instrumentation matrix behavior

- 1 Use `strategy.fail-fast: false` so all batches complete and report their failures in one workflow run.
- 2 Start one fresh API-35 emulator per matrix job.
- 3 Download the APK bundle built by `build-apks`.
- 4 Install the target and instrumentation APKs directly with `adb`.
- 5 Discover the registered instrumentation runner from the installed package instead of hard-coding it.
- 6 Run only the comma-separated classes assigned to that matrix entry.
- 7 Capture the complete instrumentation output.
- 8 Fail the job when output contains `FAILURES!!!`, `INSTRUMENTATION_FAILED`, an uncaught process crash, or lacks the expected `OK` (completion marker).
- 9 Capture filtered logcat and a screenshot on failure.
- 10 Upload uniquely named artifacts such as `instrumentation-categories`.

Recommended workflow controls

```
concurrency:
  group: keepriva-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

strategy:
  fail-fast: false
  max-parallel: 5
```

`cancel-in-progress` avoids spending emulator minutes on an outdated commit after a newer commit is pushed.

Visual-verification job

The visual job should run in parallel with the instrumentation matrix and must not rerun the complete instrumentation suite.

Its responsibilities are:

- 1 Install the APKs from the build artifact.
- 2 Capture deterministic setup, home, and real unlock screenshots.
- 3 Validate the corresponding UI hierarchy dumps.
- 4 Run only the optional biometric diagnostic test.
- 5 Upload screenshots, UI dumps, package diagnostics, and biometric logs.

Later phases will add browser, search, item-details, and history screenshots to this job.

Verification gate

Add a final lightweight job that depends on:

- `build-apks`.
- Every instrumentation matrix result.
- `visual-verification`.

The gate succeeds only when the build and every blocking matrix batch succeed. The biometric diagnostic remains informational.

Use the gate job as the required branch-protection check, rather than requiring every dynamically named matrix job individually.

Test tasks

- 1 Prove every existing test class appears in exactly one blocking batch, except the biometric diagnostic.
- 2 Compare the total executed test count with the baseline.
- 3 Intentionally fail one test in a disposable branch and verify:
- 4 Its logical batch fails.
- 5 Other batches continue because `fail-fast` is false.
- 6 The final gate fails.
- 7 The failing method is identifiable from the batch log.
- 8 Verify matrix jobs use the exact same APK bytes by comparing checksums.
- 9 Verify app data is isolated because each batch uses a fresh runner/emulator.

Exit criteria

- The same 88 normal tests pass in parallel batches.
- The optional biometric diagnostic remains non-blocking.
- APKs are built once per workflow.
- No matrix job recompiles the application.
- The final gate accurately represents the complete workflow result.
- Parallel wall-clock time is measured over at least two green runs.

Rebalancing rule

After two green runs, rebalance only if one batch consistently takes more than 30 percent longer than the median batch. Keep related classes together unless the time difference is material.

Long-run safety net

Keep a manual or weekly full serial instrumentation workflow. Parallel isolation is the normal push/PR gate; the serial run detects accidental order dependence or leaked global state.

Phase 2 - Introduce the screen router and reusable UI foundation

Objective

Create the navigation and component foundation without changing user-visible functionality significantly.

Production tasks

- 1 Add a root content container owned by `MainActivity`.
- 2 Add `VaultScreenRouter` with explicit screen states.
- 3 Define navigation state objects for:
 - 4 Current category.
 - 5 Search query.
 - 6 Selected item ID.
 - 7 Selected history-version ID.
 - 8 List scroll position.
- 9 Extract reusable toolbar, empty-state, category-row, item-row, field-row, and action-row components.
- 10 Add a `RecyclerView` dependency and stable-ID adapters.
- 11 Keep `SecretKey` and security lifecycle behavior in `MainActivity`.
- 12 Clear the router stack and all sensitive screen state when the vault locks.
- 13 Add a screen-level loading/error state so database failures do not leave an empty or half-rendered screen.
- 14 Keep existing home behavior available until Phase 3 is complete.

Test tasks

Add navigation/lifecycle coverage for:

- Router starts at vault browser after successful unlock.
- Back navigation restores the previous screen.
- Lock from any screen returns to unlock.
- Activity recreation never restores decrypted item values without unlocking.
- Screen stack is empty after explicit lock, auto-lock, or screen-off lock.
- No vault key or decrypted item is placed in a `Bundle`.

Update `KeeprivaLifecycleRobustnessTest` and add pure JVM tests for router-state transitions where possible.

CI batch

- Instrumentation: `auth-lifecycle-smoke`.
- Pure routing-state tests: JVM tests in `build-apks`.

Exit criteria

- Navigation infrastructure exists without changing persisted data.
- All old tests still pass or have equivalent semantic updates.
- Locking clears navigation and sensitive screen state.
- Back navigation has deterministic tests.

Phase 2A - Modularize MainActivity without changing behavior

Objective

Reduce the 3,684-line `MainActivity` to a small, auditable lifecycle and composition shell before the browser, search, details, move, history, and backup features are expanded. This phase changes source ownership and event routing only. It must not intentionally change visible wording, accessibility identifiers, database content, encryption, backup formats, navigation behavior, or feature behavior.

Why Phase 2A is required here

Phase 2 provides the stable activity root and router needed for extraction. Phase 3 begins the first large user-visible redesign. Extracting now avoids implementing every later feature inside the monolith and avoids simultaneously debugging structural moves and intentional UX changes.

Target responsibilities retained by MainActivity

`MainActivity` retains only:

- 1 Android lifecycle callbacks and receiver registration.
- 2 The activity-owned root content container.
- 3 Construction and teardown of the activity-scoped `VaultSessionCoordinator`, which solely owns the in-memory `SecretKey`.
- 4 Construction and teardown of session-scoped feature controllers.
- 5 Router coordination and screen dispatch.
- 6 Activity-result entry points delegated to the transfer coordinator.
- 7 Explicit lock, auto-lock, screen-off lock, and destruction cleanup.
- 8 Small package-private diagnostics required by lifecycle instrumentation tests.

It must not contain category-tree construction, item-editor construction, export-format selection, backup/restore dialogs, security-settings dialogs, or generic view-factory methods after the corresponding extraction wave completes.

Target component design

```
MainActivity
|-- VaultSessionCoordinator
|-- VaultRootRenderer
|-- VaultScreenRouter
|-- ActivityResultCoordinator
|-- SetupController
|-- UnlockController
|-- SecuritySettingsController
|-- LegacyVaultBrowserController
|-- CategoryManagementController
|-- ItemDialogController
`-- DataTransferController

Controllers
|-- receive immutable view state
|-- emit typed callbacks
|-- own only current-screen views/dialogs
`-- release references on lock or screen replacement

Repositories/gateways
|-- contain database operations
|-- contain hierarchy/search rules
`-- never own Activity views
```

Mandatory dependency rules

- 1 Controllers depend on narrow interfaces, not directly on other controllers.
- 2 UI controllers never store `SecretKey` or complete decrypted item collections in static or persisted state.
- 3 Adapters continue receiving presentation-only row models.
- 4 `Context` references are activity scoped and released by `close()/destroy()` methods.
- 5 Dialog ownership is centralized so lock and teardown can dismiss every open dialog deterministically.
- 6 `VaultDatabase`, crypto managers, backup/import/export managers, and clipboard security remain independent of UI controllers.
- 7 Controller callbacks return stable IDs and user intent; the activity/session gateway performs sensitive operations.
- 8 No controller uses `Bundle`, `Parcelable`, Java serialization, or saved-instance state for decrypted values.
- 9 Phase 2 semantic content descriptions and all existing test selectors remain unchanged.

- 10 No new framework, dependency-injection container, fragment migration, or Compose migration is introduced.

Extraction waves

Wave A - Contracts, lifecycle cleanup, and common UI

- 1 Add controller lifecycle contracts such as `VaultController` and `DismissibleUiOwner`.
- 2 Add typed action interfaces for setup, unlock, browser, item, category, security, and transfer events.
- 3 Add `VaultRootRenderer` around the Phase 2 root container.
- 4 Add `DialogRegistry` that can dismiss and forget every registered dialog on lock/destruction.
- 5 Move generic view construction into `VaultViewFactory/VaultUiComponents`.
- 6 Add architecture tests that prohibit sensitive/persistable controller fields.

For reviewability, Wave A may be split into A1 and A2. A1 establishes key ownership, root rendering, the controller teardown contract/registry, and their JVM tests. A2 adds the dialog registry, typed feature-action contracts, and common view factory before any feature controller is extracted.

Wave B - Setup, unlock, session, and security settings

- 1 Extract setup rendering and validation-event wiring into `SetupController`.
- 2 Extract unlock rendering, progress state, and biometric button events into `UnlockController`.
- 3 Keep PBKDF2, key unwrap/migration, and key assignment in the activity-owned session coordinator.
- 4 Extract preferences, reauthentication, auto-lock, clipboard-timeout, password-change, and biometric-settings dialogs into `SecuritySettingsController`.
- 5 Preserve the existing background executor and ensure callbacks are ignored after controller teardown.

Wave C - Browser, category management, and item dialogs

- 1 Move current home/tree rendering unchanged into `LegacyVaultBrowserController`.
- 2 Move category path/depth validation into `CategoryHierarchyService` with JVM tests.
- 3 Move category-management dialogs and force-delete/move-content event wiring into `CategoryManagementController`.
- 4 Move the existing details/editor/delete/Undo dialog workflow into `ItemDialogController`.
- 5 Preserve all existing strings, semantic identifiers, category tombstones, selection rules, and dialog-root behavior.
- 6 Mark legacy controllers clearly so Phases 3 and 5 replace rather than extend them.

Wave D - Import, export, backup, and Activity results

- 1 Move import/export/template/backup/restore workflow state into a session-scoped `DataTransferController`.
- 2 Add `ActivityResultCoordinator` to map existing request codes to typed transfer callbacks.
- 3 Keep pending sensitive bytes memory-only and clear them on cancellation, completion, lock, destruction, and timeout.
- 4 Preserve all existing document-picker intents, MIME types, suggested filenames, dialog roots, and reauthentication behavior.

Wave E - Final wiring and dead-code removal

- 1 Compose controllers in one place after release-security verification succeeds.
- 2 Add one idempotent `clearSessionAndControllers()` path used by every lock/destruction route.
- 3 Remove duplicate helpers and forwarding methods after callers have migrated.
- 4 Verify `MainActivity` contains no feature-specific dialog construction.
- 5 Record final line counts and component ownership in the Phase 2A implementation report.

Test tasks

Add fast JVM tests for:

- Controller event-to-action mapping.
- Category hierarchy/path/depth rules.
- Controller teardown and dialog-registry idempotence.
- Activity-result request-code dispatch.
- Controller field-type security rules.
- No key, decrypted model, `Bundle`, `Parcelable`, or `Serializable` retention.

Add focused instrumentation tests for:

- Setup, unlock, browser, and lock roots after controller extraction.
- Explicit lock, auto-lock, screen-off lock, and recreation cleanup.
- Open-dialog dismissal during lock and Activity destruction.
- Late background-unlock callback suppression after teardown.
- Import/export/backup Activity-result delegation.
- Back and router behavior remaining equivalent.

Run all existing 95 instrumentation tests unchanged wherever their semantic behavior is unchanged. Test changes are allowed only for a deliberate ownership/test-hook adjustment, never to weaken an assertion.

CI placement

- Controller and architecture JVM tests run in `build-apks` through `testDebugUnitTest`.
- New lifecycle/wiring instrumentation belongs to `auth-lifecycle-smoke`.
- Existing feature tests remain in their Phase 2 batches.
- Update explicit expected-count guards in the same commit as any added test.
- Run the manual serial safety net before declaring Phase 2A complete because extraction can reveal order-dependent leaked state.

Quantitative exit criteria

- `MainActivity` is reduced from 3,684 lines to at most 1,200 non-generated lines.
- No feature controller exceeds 700 lines; split larger controllers by workflow.
- No controller has a static `Activity`, `View`, `Dialog`, `SecretKey`, or decrypted model field.
- Every controller with UI ownership has an idempotent teardown method.
- The database remains version 3.
- The backup format remains version 1.
- The manifest still has no `INTERNET` permission.
- All 95 existing instrumentation tests and all nine Phase 2 JVM tests pass before counting new Phase 2A tests.
- All blocking matrix jobs, visual verification, and the manual serial safety net are green.
- Setup, unlock, home, category, item, security, import/export, backup/restore, and lock behavior remain visually and semantically equivalent.

Rollback point

Return to the Phase 2 baseline tag created from the latest green `ui_eh_ph02_screen_routing` head. The underlying Phase 2 code baseline is `2212df03ad713e5693bfd8d6b46b7d5ae926b363`. Phase 2A changes no persisted format, so rollback requires no database or backup conversion.

Downstream contract

Every later phase must replace or extend the appropriate controller, renderer, service, or repository. No later phase may move feature behavior back into `MainActivity`.

Phase 3 - Redesign the vault browser

Objective

Replace the long expandable tree with a category-first browser inspired by the useful navigation concepts in KeePass2Android.

Production tasks

- 1 Replace `LegacyVaultBrowserController` with `VaultBrowserController`; do not reintroduce browser rendering in `MainActivity`.
- 2 Add immutable `VaultBrowserViewState` and `VaultBrowserActions` contracts. The controller receives presentation rows and emits category/item IDs and toolbar intents.
- 3 Add a top toolbar containing:
 - 4 Current vault/category title.
 - 5 Search.
 - 6 Lock.
 - 7 Overflow menu.
- 8 Use `CategoryHierarchyService` from Phase 2A to build breadcrumbs and validate direct children.
- 9 Render direct subcategories in a `Subcategories` section.
- 10 Render direct items in an `Items` section through the stable-ID adapters introduced in Phase 2.
- 11 Use compact rows with semantic category/item icons.
- 12 Show item title and full category path; do not expose password or notes.
- 13 Add a floating `+` action with `Entry` and `Subcategory` options.
- 14 Route import, export, backup, preferences, security, and category management overflow intents to the Phase 2A coordinators.
- 15 Preserve selected category and scroll position in `VaultNavigationState` when returning from details.
- 16 Preserve category-depth rules and built-in-category tombstones.
- 17 Keep empty states for:
 - 18 Empty vault.
 - 19 Empty category.
 - 20 Category with subcategories but no direct items.

Test tasks

Update existing tests that currently expect inline expansion:

- `KeepivaAuthHomeTest`.
- `KeepivaCategoryPreferencesTest`.
- `KeepivaItemCrudTest`.
- `KeepivaUiSmokeTest`.

Add/replace tests for:

- Built-in categories appear as browser rows.
- Custom root categories appear.
- Opening a category shows only direct children and direct items.
- Breadcrumb shows the complete category path.
- Back returns to the parent category.
- Item count remains correct.
- Category icons retain their semantic families.
- Floating add menu offers `Entry` and `Subcategory`.

- Overflow actions remain reachable.
- Empty-category state is visible.
- Lock remains accessible from the toolbar.

CI batch

- `categories` for hierarchy and category management.
- `item-core` for category-to-item navigation.
- `auth-lifecycle-smoke` for toolbar and reachability checks.

Visual artifacts

Add screenshots for:

- Root vault browser.
- A nested category.
- Empty category.

Exit criteria

- No item is lost or moved by the UI redesign.
 - All previous category CRUD and deletion behavior remains green.
 - The home screen no longer depends on expanding the complete tree.
 - Management tools remain accessible but do not dominate the vault content.
 - `LegacyVaultBrowserController` is removed and `MainActivity` contains no browser-specific rendering code.
-

Phase 4 - Add the dedicated search-results screen

Objective

Move search results out of the category hierarchy into a separate compact list.

Production tasks

- 1 Add `SearchResultsController`, `SearchResultsViewState`, and `SearchResultsActions`; `MainActivity` only routes to the controller.
- 2 Extract matching/normalization into `VaultSearchService` with no Android view dependencies.
- 3 Search toolbar action navigates to the explicit `SEARCH_RESULTS` route.
- 4 Add a focused search input and clear-query action.
- 5 Reuse the current search fields:
- 6 Title.
- 7 Category.
- 8 Username/email.
- 9 Phone numbers.
- 10 Website/app name.
- 11 Website URL.
- 12 Notes.
- 13 Custom-field names and values.
- 14 Continue excluding passwords from search indexing.
- 15 Return only matching items, not category branches.
- 16 Each row displays only:
- 17 Item icon.

- 18 Item title.
- 19 Full category path.
- 20 Do not show sensitive snippets explaining why an item matched.
- 21 Sort results deterministically: normalized title, then category path, then item ID.
- 22 Add result count, empty query, and no-results states.
- 23 Opening a result navigates to full-screen item details through the router.
- 24 Back restores the prior query and result-list position from in-memory navigation state.
- 25 Debounce filtering only if measurement shows it is needed; do not introduce arbitrary test sleeps.

Test tasks

Create `KeeprivaSearchResultsTest` with cases for:

- Search opens as a separate screen.
- Title match shows title and full category path.
- Username match returns the item without exposing username in a result snippet unless deliberately approved.
- Notes match returns the item without displaying notes.
- Custom-field value match returns the item without exposing the value.
- Password text does not produce a result.
- Matching is case-insensitive.
- Nonmatching items are absent.
- No-results state appears.
- Clear-query resets the list.
- Opening a result shows the correct item.
- Back restores query and position.

Move obsolete inline-search assertions out of `KeeprivaItemCrudTest` and `KeeprivaCategoryPreferencesTest`.

Add pure JVM tests for search normalization and matching. Keep only screen behavior in emulator tests.

CI batch

Rename/expand `item-core` to `item-core-search`:

```
KeeprivaItemCrudTest
KeeprivaSearchResultsTest
```

Visual artifacts

- Search with results.
- Search with no results.

Exit criteria

- Search is no longer rendered inside the category tree.
- Result rows contain item title and category path.
- Search does not reveal password or sensitive snippets.
- Search logic has fast JVM coverage and focused UI coverage.
- Search logic and rendering remain outside `MainActivity`.

Phase 5 - Full-screen item details, timestamps, and secure copy actions

Objective

Replace the item-details dialog with a professional full-screen view and add secure copying for every displayed value.

Production tasks

- 1 Replace the Phase 2A `ItemDialogController` details path with `ItemDetailsController`, `ItemDetailsViewState`, and `ItemDetailsActions`.
- 2 Create a full-screen item-details view with:
- 3 Back navigation.
- 4 Item title.
- 5 Edit action.
- 6 Overflow actions for move, export, and delete.
- 7 Render reusable field rows for every non-empty value.
- 8 Keep passwords masked initially with separate reveal/hide and copy controls.
- 9 Add copy actions for:
- 10 Category path.
- 11 Username/email.
- 12 Password.
- 13 Phone 1, 2, and 3.
- 14 Website/app name.
- 15 Website URL.
- 16 Notes.
- 17 Every custom field.
- 18 Created timestamp.
- 19 Modified timestamp.
- 20 Emit copy intents through `ItemDetailsActions`; the session-owned clipboard gateway invokes `ClipboardSecurityManager.copySensitive()`.
- 21 Respect the configured clipboard-clear timeout.
- 22 Show a short confirmation including the timeout, without repeating the copied value.
- 23 Format `createdAt` and `updatedAt` through a testable formatter before creating view state.
- 24 Preserve existing sensitive custom-field masking.
- 25 Ensure locking or leaving the vault clears Keepiva-owned clipboard content according to current security rules.

Test tasks

Create `KeepivaClipboardActionsTest` and update `KeepivaItemCrudTest`.

Required tests:

- Details open as a full-screen view, not a dialog.
- Password is masked initially.
- Reveal/hide remains functional.
- Every populated field exposes a correctly labelled copy action.
- Empty fields do not create blank rows or copy buttons.
- Standard and custom fields use the secure clipboard manager.
- Copy confirmation does not expose the copied secret.
- A newer clipboard value is not erased by an older delayed-clear task.
- Created timestamp is stable after editing.

- Modified timestamp increases after editing.
- Lock from details returns to unlock and removes sensitive details from the hierarchy.

Clipboard timing and ownership logic should remain covered mainly by JVM/unit tests where practical. UI tests should verify wiring and accessibility rather than waiting 15-60 seconds.

CI batch

Add a future batch:

```
item-actions
  KeeprivaClipboardActionsTest
```

Keep core CRUD in `item-core-search`.

Visual artifacts

- Item details with standard fields.
- Masked password row.
- Item details with custom fields.

Exit criteria

- The old details dialog is removed.
- Every displayed value has a deterministic secure copy action.
- Created and modified timestamps are visible and correct.
- No raw secret appears in toast text, logcat, or accessibility descriptions.
- The old details branch is removed from `ItemDialogController`; details behavior is not moved into `MainActivity`.

Phase 6 - Add explicit item movement

Objective

Provide a clear Move action independent of full item editing.

Production tasks

- 1 Add `Move item` to `ItemDetailsActions` and route to `MOVE_ITEM`.
- 2 Add `ItemMoveController`, `ItemMoveViewState`, and `ItemMoveActions`.
- 3 Use `CategoryHierarchyService` to create a hierarchical destination list with full category paths.
- 4 Mark the current category and disable a no-op move.
- 5 Exclude hidden/deleted built-in categories.
- 6 Validate the destination again in `ItemRepository.moveItem()` at commit time.
- 7 Update only the item's category.
- 8 Preserve item ID, creation time, and all other values.
- 9 Update modification time.
- 10 Perform the operation transactionally.
- 11 Return through the router to item details and display the new category path.
- 12 When returning to the browser, open the destination category.
- 13 Add a confirmation only if needed for clarity; moving itself is reversible through the later version-history feature.

Test tasks

Create `KeeprivaItemMoveTest` with cases for:

- Move action is available from details.
- Destination list contains built-in and custom categories.
- Nested destinations show full paths.
- Current category cannot be selected as a meaningful move.
- Cancel leaves the item unchanged.
- Move preserves every field and the item ID.
- Move preserves creation time.
- Move updates modification time.
- Item disappears from the old category and appears in the destination.
- Move to a nested custom category works.
- Hidden/deleted category is not selectable.
- Database failure produces no partial move.

Until Phase 7 is present, explicitly mark the version-snapshot assertion as pending; add it immediately in Phase 7.

CI batch

Add `KeeprivaItemMoveTest` to `item-actions`.

Exit criteria

- Item movement is discoverable without editing all fields.
- Moves are atomic and preserve item data.
- The browser and search results immediately reflect the new path.
- Move UI and persistence are isolated from `MainActivity` behind controller/repository contracts.

Phase 7 - Add encrypted item-version storage and v3-to-v4 migration

Objective

Add a safe data foundation for version history before exposing history UI.

Database design

Increase database version from 3 to 4 and add `vault_item_versions`.

Suggested logical columns:

<code>id</code>	<code>INTEGER PRIMARY KEY AUTOINCREMENT</code>
<code>item_id</code>	<code>INTEGER NOT NULL</code>
<code>version_number</code>	<code>INTEGER NOT NULL</code>
<code>encrypted_snapshot</code>	<code>TEXT NOT NULL</code>
<code>captured_at</code>	<code>INTEGER NOT NULL</code>

Add indexes for:

```
(item_id, version_number DESC)
(item_id, captured_at DESC)
```

The encrypted snapshot contains:

- Complete prior item values.
- Prior `createdAt` and `updatedAt`.
- Change reason: edited, moved, or restored.
- Changed field names.

Do not store field values or field-change details in plaintext metadata.

Production tasks

- 1 Add the explicit sequential `migrate3To4()` path.
- 2 Update schema verification to require the history table and indexes.
- 3 Add `VaultItemVersion` model.
- 4 Extend the Phase 2A repository boundary with `ItemRepository` and `ItemVersionRepository`; no migration or snapshot SQL belongs in an Activity or UI controller.
- 5 When an existing item changes:
 - 6 Read the current item.
 - 7 Compare normalized values.
 - 8 If nothing changed, do not create a version.
 - 9 Otherwise insert the encrypted current snapshot.
 - 10 Update the current item.
 - 11 Commit both operations in one transaction.
- 12 Capture versions for edits and category moves.
- 13 Add a retention policy of 20 versions per item.
- 14 Delete only the oldest excess versions after a successful new snapshot, in the same transaction.
- 15 Define deletion behavior:
 - 16 Permanent item deletion deletes its versions.
 - 17 Immediate Undo must restore the item and its captured versions.
- 18 Force-delete of a category subtree deletes corresponding histories atomically.
- 19 Keep imports without history valid.

Test tasks

Create `KeeprivaItemVersionMigrationTest` and repository-level instrumentation tests.

Required tests:

- Fresh install creates schema v4.
- v3 database upgrades to v4 without item/category loss.
- Migration history records v3 to v4 once.
- Existing timestamps and encrypted payloads remain readable.
- Editing creates exactly one previous version.
- Moving creates exactly one previous version.
- Saving unchanged data creates no version.
- Failed update creates neither a partial version nor a partial current update.
- Snapshots decrypt only with the correct vault key.
- Version ordering is deterministic.
- The 21st version removes only the oldest snapshot.
- Permanent deletion removes history.
- Delete then immediate Undo restores current item and history.
- Category force-delete removes histories belonging to deleted items.
- Database downgrade remains rejected.

Add pure JVM tests for change detection, changed-field naming, and retention selection.

CI batch

Add a dedicated `history-storage` matrix batch.

This batch must remain separate from ordinary UI tests so schema/migration failures are immediately visible.

Exit criteria

- The v3-to-v4 migration is non-destructive and repeat-safe.
 - Every modifying operation is atomic with its snapshot.
 - No-op saves create no history noise.
 - History retention is deterministic.
 - All encrypted data remains unreadable without the vault key.
 - Version persistence is testable independently of screen controllers and `MainActivity` remains schema unaware.
-

Phase 8 - Add history list and version-detail UI

Objective

Expose item history without weakening password masking or clipboard security.

Production tasks

- 1 Extend `ItemDetailsController` with a non-sensitive version summary section.
- 2 Add `ItemHistoryController`, `ItemVersionDetailsController`, immutable view states, and typed action interfaces.
- 3 Show versions newest first with:
 - 4 Date and time.
 - 5 Change reason.
 - 6 Changed field names.
- 7 Show a clear no-history state.
- 8 Navigate to a dedicated read-only version-details route.
- 9 Display the complete historical snapshot using the same field-row component as current details.
- 10 Keep historical passwords and sensitive custom fields masked initially.
- 11 Reuse the session-owned secure-copy gateway and timeout behavior.
- 12 Add `Restore this version`.
- 13 Before restoring:
 - 14 Confirm the action.
 - 15 Verify the target item still exists.
 - 16 Validate the historical category still exists.
- 17 If the historical category no longer exists, route through the existing move/destination component rather than duplicating category-selection logic.
- 18 Restore all historical values through `ItemVersionRepository` while preserving current item ID and original creation date.
- 19 Snapshot the current item before restoration.
- 20 Set the restored current item's modification time to now.
- 21 Add `Remove this version` with confirmation.
- 22 Removing a historical version must not change the current item.

Test tasks

Create `KeepivaItemHistoryTest` with cases for:

- No-history state is displayed.

- Edit creates a dated history row.
- Move creates a history row with category change.
- Version rows are newest first.
- Changed-field names are correct and values are not shown in the list.
- Opening a row shows the correct historical snapshot.
- Historical password is masked initially.
- Historical values use secure copy actions.
- Cancel restore leaves current item unchanged.
- Restore updates current values and creates a snapshot of the replaced state.
- Restore preserves item ID and original creation time.
- Restore updates modification time.
- Missing historical category requires destination selection.
- Cancel remove keeps the version.
- Confirm remove deletes only the selected version.
- Back returns to the same history-list position.
- Lock from history/version details clears sensitive UI and returns to unlock.

CI batch

Add `KeepPrivaItemHistoryTest` to `history-storage`, unless runtime measurement requires a separate `history-ui` batch.

Visual artifacts

- Item details with previous versions.
- History list.
- Historical-version details with masked password.
- Restore confirmation.

Exit criteria

- Historical data is inspectable without exposing secrets by default.
 - Restore and remove actions are transactional and independently tested.
 - Missing-category restoration cannot create invalid data.
 - History controllers release decrypted historical view state on Back, lock, replacement, and destruction.
-

Phase 9 - Add version-aware backup and restore

Objective

Ensure encrypted backups preserve history while remaining backward compatible.

Production tasks

- 1 Extend the Phase 2A `DataTransferController`; do not add backup dialogs or pending buffers back to `MainActivity`.
- 2 Increase encrypted backup format from 1 to 2.
- 3 Add item-history records to the encrypted backup payload.
- 4 Keep the entire logical payload encrypted under the backup-password-derived key.
- 5 Update restore validation for:
- 6 History item references.

- 7 Duplicate version IDs/numbers.
- 8 Invalid timestamps.
- 9 Invalid or oversized history arrays.
- 10 Restore categories, current items, and history through repository transactions.
- 11 Continue accepting format-v1 backups.
- 12 Treat v1 backups as current-state-only backups with empty history.
- 13 Reject unsupported future backup versions with a clear message.
- 14 Include history only in encrypted `.pvault` backups.
- 15 Keep TXT, HTML, PDF, and ordinary JSON exports current-state-only.
- 16 Document that plaintext/export formats intentionally exclude old credentials.

Test tasks

Create `KeepivaVersionedBackupTest` and extend `KeepivaDataTransferTest`.

Required tests:

- Format-v2 backup contains current items and versions after decryption.
- Correct password round-trip restores version history.
- Wrong password restores nothing.
- Tampered history payload fails integrity/validation.
- Restore is atomic when a history reference is invalid.
- Format-v1 backup still restores successfully with empty history.
- Unsupported future format is rejected.
- TXT/HTML/PDF/ordinary JSON do not contain historical credentials.
- Encrypted backup does not contain plaintext current or historical secrets.
- Restore preserves version ordering and timestamps.
- Backup password arrays and plaintext buffers are cleared according to current security practice.

CI batch

Expand `data-transfer`:

```
KeepivaDataTransferTest
KeepivaVersionedBackupTest
```

If this batch becomes more than 30 percent slower than the median, split it into:

- `data-transfer-ui`.
- `backup-compatibility`.

Exit criteria

- New encrypted backups preserve version history.
- Old encrypted backups remain compatible.
- Plaintext exports never include history.
- Invalid restores never partially replace the vault.
- Pending transfer secrets remain owned and cleared by `DataTransferController`, including every Activity-result exit path.

Phase 10 - Integration, accessibility, visual QA, and release gate

Objective

Verify the redesigned application as one coherent product and eliminate obsolete UI/test paths.

Production tasks

- 1 Remove obsolete expandable-tree rendering and hidden list-container compatibility code.
- 2 Remove obsolete item-details dialog code.
- 3 Remove `LegacyVaultBrowserController`, the legacy item-dialog paths, and temporary compatibility callbacks.
- 4 Remove temporary adapters/helpers replaced by final screen components.
- 5 Run architecture checks proving that `MainActivity` is still a thin composition/lifecycle shell and no controller violates the Phase 2A sensitive-field rules.
- 6 Review typography, spacing, icons, touch targets, empty states, and error states.
- 7 Verify small phone, Pixel 6, landscape, and large-font behavior.
- 8 Verify all screens remain protected from screenshots in real non-CI builds.
- 9 Confirm the manifest still has no `INTERNET` permission.
- 10 Review log statements and error messages for secret leakage.
- 11 Update user documentation and release notes.

Test tasks

- 1 Run every parallel batch.
- 2 Run the manual/weekly full serial suite.
- 3 Run database migration tests from realistic v1, v2, and v3 fixtures where available.
- 4 Run encrypted backup v1 and v2 compatibility tests.
- 5 Run accessibility checks for labels, focus order, and 48dp touch targets.
- 6 Capture final screenshots for:
 - 7 Setup.
 - 8 Unlock.
 - 9 Root browser.
 - 10 Nested category.
 - 11 Search results.
 - 12 Item details.
 - 13 Move item.
 - 14 History list.
 - 15 Version details.
- 16 Compare screenshots with approved design baselines.
- 17 Verify no test uses arbitrary long sleeps when an explicit condition can be observed.
- 18 Verify no test depends on execution order or state left by another test.

Exit criteria

- All blocking matrix batches pass.
- The serial regression suite passes.
- Lint and JVM unit tests pass.
- Screenshot/UI hierarchy verification passes.
- Backup compatibility and database migration gates pass.
- No release build security behavior was weakened for testing.
- `MainActivity` remains within the Phase 2A responsibility boundary and contains no feature-specific UI construction.
- The branch is ready for review and merge.

8. Final logical emulator-batch design

After all phases, use the following normal push/PR matrix.

Batch	Classes	Expected focus
auth-lifecycle-smoke	KeeprivaAuthHomeTest, KeeprivaLifecycleRobustnessTest, KeeprivaUiSmokeTest	Setup, unlock, lock, recreation, global reachability
categories	KeeprivaCategoryPreferencesTest, KeeprivaCategoryDeletionTest	Category browser, hierarchy, preferences, deletion
item-core-search	KeeprivaItemCrudTest, KeeprivaSearchResultsTest	CRUD, browser-to-item navigation, dedicated search
item-actions	KeeprivaClipboardActionsTest, KeeprivaItemMoveTest	Field copy actions and category movement
history-storage-ui	KeeprivaItemVersionMigrationTest, KeeprivaItemHistoryTest	Schema v4, snapshots, restore/remove, history UI
data-transfer	KeeprivaDataTransferTest, KeeprivaVersionedBackupTest	Import/export and backup compatibility
security	KeeprivaSecuritySettingsTest	Reauthentication, auto-lock, clipboard policy, password, biometric settings

KeeprivaBiometricCiTest remains in the non-blocking visual/biometric diagnostic job.

The green Phase 2 starting counts are `auth-lifecycle-smoke=32`, `categories=24`, `item-core=14`, `data-transfer=12`, and `security=12`, for 94 blocking tests. Phase 2A and every later phase must update the relevant explicit batch guard in the same commit that adds tests. The diagnostic test remains outside the blocking total.

Why classes are grouped this way

- A failed category test does not obscure an item-history failure.
- Database migration/history has its own diagnostic boundary.
- Data-transfer failures are isolated from ordinary item CRUD.
- Security settings remain independent from field-level clipboard UI tests.
- New feature classes can grow without making the original item CRUD class unmaintainable.

Maximum parallelism

Start with `max-parallel: 5`. Seven matrix entries may execute in two waves if runner concurrency is limited. Increase only after observing queue time, runner availability, and total billed minutes.

Parallel execution reduces workflow wall-clock time but increases total emulator runner-minutes because every batch boots a separate emulator. The target is faster developer feedback with acceptable CI cost, not the smallest possible total compute usage.

9. Proposed GitHub Actions matrix blueprint

This is a design blueprint, not a drop-in replacement. Exact paths and runner detection must be validated in the implementation phase.

```
jobs:
  build-apks:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: "17"
      - uses: gradle/actions/setup-gradle@v4
        with:
```

```

    gradle-version: "9.1.0"
  - name: Install Android build components
    run: sdkmanager "platforms;android-36" "build-tools;36.0.0" "platform-tools"
  - name: Compile, lint, unit test, and build APKs
    run: |
      gradle \
        :app:lintDebug \
        :app:testDebugUnitTest \
        :app:assembleDebug \
        :app:assembleDebugAndroidTest \
        --stacktrace
  - name: Stage APK bundle and checksums
    run: bash scripts/ci/stage-apks.sh
  - uses: actions/upload-artifact@v4
    with:
      name: keepriya-apks-${{ github.sha }}
      path: ci-apks/**
      retention-days: 7

instrumentation:
  needs: build-apks
  runs-on: ubuntu-latest
  timeout-minutes: 18
  strategy:
    fail-fast: false
    max-parallel: 5
  matrix:
    include:
      - id: auth-lifecycle-smoke
        classes: >-
          com.example.privatevault.KeepriyaAuthHomeTest,
          com.example.privatevault.KeepriyaLifecycleRobustnessTest,
          com.example.privatevault.KeepriyaUiSmokeTest
      - id: categories
        classes: >-
          com.example.privatevault.KeepriyaCategoryPreferencesTest,
          com.example.privatevault.KeepriyaCategoryDeletionTest
      - id: item-core-search
        classes: >-
          com.example.privatevault.KeepriyaItemCrudTest,
          com.example.privatevault.KeepriyaSearchResultsTest
      - id: item-actions
        classes: >-
          com.example.privatevault.KeepriyaClipboardActionsTest,
          com.example.privatevault.KeepriyaItemMoveTest
      - id: history-storage-ui
        classes: >-
          com.example.privatevault.KeepriyaItemVersionMigrationTest,
          com.example.privatevault.KeepriyaItemHistoryTest
      - id: data-transfer
        classes: >-
          com.example.privatevault.KeepriyaDataTransferTest,
          com.example.privatevault.KeepriyaVersionedBackupTest
      - id: security
        classes: com.example.privatevault.KeepriyaSecuritySettingsTest

steps:
  - uses: actions/checkout@v4
  - uses: actions/download-artifact@v4
    with:
      name: keepriya-apks-${{ github.sha }}
      path: ci-apks
  - name: Verify downloaded APK checksums
    run: sha256sum --check ci-apks/SHA256SUMS
  - name: Boot emulator and run batch
    uses: reactivecircus/android-emulator-runner@v2
    with:
      api-level: 35
      arch: x86_64
      profile: pixel_6
      target: google_api
      disable-animations: true
      emulator-options: >-
        -no-window
        -no-snapshot
        -wipe-data
        -gpu swiftshader_indirect
        -noaudio
        -no-boot-anim
        -camera-back none

```

```

      script: |
        bash scripts/ci/run-instrumentation-batch.sh \
          "${{ matrix.id }}" \
          "${{ matrix.classes }}"
    - name: Upload batch diagnostics
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: instrumentation-${{ matrix.id }}
        path: ci-artifacts/${{ matrix.id }}/**
        retention-days: 14

visual-verification:
  needs: build-apks
  runs-on: ubuntu-latest
  timeout-minutes: 18
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with:
        name: keepriya-apks-${{ github.sha }}
        path: ci-apks
    - name: Boot emulator and capture deterministic UI states
      uses: reactivecircus/android-emulator-runner@v2
      with:
        api-level: 35
        arch: x86_64
        profile: pixel_6
        target: google_apis
        disable-animations: true
        script: bash scripts/ci/run-visual-verification.sh
    - name: Upload visual diagnostics
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: visual-verification
        path: ci-artifacts/visual/**
        retention-days: 14

verification-gate:
  if: always()
  needs:
    - build-apks
    - instrumentation
    - visual-verification
  runs-on: ubuntu-latest
  steps:
    - name: Enforce complete verification result
      run: |
        test "${{ needs.build-apks.result }}" = "success"
        test "${{ needs.instrumentation.result }}" = "success"
        test "${{ needs.visual-verification.result }}" = "success"

```

10. CI helper-script requirements

Move complex shell logic out of YAML and into version-controlled scripts.

scripts/ci/stage-apks.sh

Responsibilities:

- 1 Locate exactly one debug target APK.
- 2 Locate exactly one debug instrumentation APK.
- 3 Copy both to stable filenames.
- 4 Generate `SHA256SUMS`.
- 5 Fail on missing or ambiguous APKs.

scripts/ci/run-instrumentation-batch.sh

Responsibilities:

- 1 Accept exactly two arguments: batch ID and comma-separated classes.
- 2 Configure deterministic emulator settings.
- 3 Install target and test APKs.
- 4 Detect and validate the instrumentation runner.
- 5 Clear the target package before the batch.
- 6 Run the selected classes.
- 7 Preserve complete output.
- 8 Detect test failures even if `adb shell am instrument` returns zero incorrectly.
- 9 Record test count and duration.
- 10 Capture process state, screenshot, UI dump, and filtered logcat on failure.
- 11 Never print vault passwords or other secrets.
- 12 Keep an explicit expected count for every batch and update it atomically with test additions/removals.
- 13 Validate that the sum of blocking batch counts equals the complete blocking inventory.

`scripts/ci/run-visual-verification.sh`

Responsibilities:

- 1 Create deterministic fixture data without weakening release behavior.
- 2 Capture approved screens.
- 3 Dump and validate UI hierarchies.
- 4 Run only the optional biometric diagnostic.
- 5 Keep biometric enrollment limitations non-blocking.

Shell validation

Every helper must pass:

```
bash -n
shellcheck
```

where ShellCheck is available in CI.

11. Test engineering rules for all phases

- 1 A production feature and its tests are one deliverable.
- 2 Tests must not depend on execution order.
- 3 Every test starts from explicit, isolated app state.
- 4 Prefer semantic content descriptions or resource IDs over coordinate clicks and fragile text matching.
- 5 Use `inRoot(isDialog())` only for genuine dialogs; full-screen views must use the activity root.
- 6 Avoid stateful indexed matchers when a semantic parent/child matcher is possible.
- 7 Avoid arbitrary sleeps. Wait for an observable screen marker, database result, or intent.
- 8 Do not make timing assertions against the real 15/30/60-second clipboard settings in UI tests; test timer policy below the UI.
- 9 Keep passwords, decrypted payloads, clipboard values, and backup passwords out of logs and assertion messages.
- 10 Use deterministic locale/time-zone handling for timestamp assertions.
- 11 Test dates by stored epoch values and formatted-pattern expectations, not by racing the wall clock.
- 12 Keep one accessibility/content-description contract for each screen and action.
- 13 When UI wording changes without semantic behavior changing, tests should normally remain stable.
- 14 Every database mutation test verifies both success and rollback behavior.

- 15 Migration tests must use real older-schema fixture databases, not only mocked version numbers.
 - 16 Backup compatibility tests must keep sanitized v1 and v2 fixture files under test resources.
 - 17 Controller tests use fake action interfaces and immutable view state; they must not require an emulator when Android rendering is not under test.
 - 18 Architecture tests reject static Activity/View/Dialog fields and `SecretKey`/decrypted-model fields in controllers.
 - 19 A controller teardown test is mandatory when the controller owns views, dialogs, callbacks, executors, or pending sensitive buffers.
 - 20 Refactoring tests preserve semantic behavior; moving code is not a reason to weaken assertions or replace deterministic waits with sleeps.
-

12. Branch and commit strategy

Use one short-lived branch per phase, based on the latest green phase point:

```
ui_eh_ph01_parallel_ci
ui_eh_ph02_screen_routing
ui_eh_ph02a_mainactivity_modularization
ui_eh_ph03_vault_browser
ui_eh_ph04_search_results
ui_eh_ph05_item_details_copy
ui_eh_ph06_move_item
ui_eh_ph07_version_storage
ui_eh_ph08_version_ui
ui_eh_ph09_versioned_backup
ui_eh_ph10_final_integration
```

Recommended commit ordering within each phase:

- 1 Pure model/data changes and unit tests.
- 2 UI implementation and focused instrumentation tests.
- 3 Existing-test migration.
- 4 Workflow/screenshot changes if needed.
- 5 Documentation and cleanup.

For Phase 2A, use one commit per extraction wave and require a green build after each wave. Do not combine all source moves into one unreviewable commit.

Do not mix unrelated security, branding, or release-signing changes into these phase branches.

13. Per-phase review checklist

Before merging any phase, confirm:

- [] Production scope matches the phase and does not pull in later features.
- [] Existing user data remains readable.
- [] New behavior has positive, negative, cancel, and rollback tests.
- [] Existing affected tests were deliberately updated rather than disabled.
- [] No test was weakened merely to make CI green.
- [] No sensitive values appear in logcat, screenshots, UI dumps, or failure messages.
- [] App locking clears the new screen state.
- [] Back navigation is deterministic.
- [] Accessibility descriptions are stable and unique.
- [] The correct logical emulator batch passes.
- [] All other batches pass, proving no regression.
- [] Visual artifacts are updated only when the intended UI changed.
- [] The serial safety-net run passes before a major milestone/release.

- [] Feature code was added to the owning controller/service/repository rather than `MainActivity`.
 - [] Controller teardown releases dialogs, views, callbacks, and pending sensitive buffers.
-

14. Final definition of done

The redesign is complete only when all of the following are true:

- 1 Vault browsing uses category-first, full-screen navigation.
 - 2 Search results use a separate list showing item title and full category path.
 - 3 Every displayed item value has a secure, labelled copy action.
 - 4 Item creation and modification times are visible and correct.
 - 5 Items can be moved explicitly without data loss.
 - 6 Item edits, moves, and restores create encrypted previous versions.
 - 7 Previous versions can be inspected, restored, and removed.
 - 8 Database v3 upgrades safely to v4.
 - 9 Encrypted backup v2 preserves history.
 - 10 Encrypted backup v1 remains restorable.
 - 11 Plaintext exports never contain previous-version secrets.
 - 12 All normal emulator batches pass in parallel.
 - 13 The optional biometric diagnostic remains clearly separated and non-blocking.
 - 14 The full serial safety-net suite passes.
 - 15 Visual verification covers every new primary screen.
 - 16 Release screenshot protection, auto-lock, clipboard cleanup, and offline-only behavior remain intact.
 - 17 `MainActivity` remains the small lifecycle/composition shell established by Phase 2A.
 - 18 Feature controllers obey sensitive-state and teardown contracts.
-

15. Recommended first action when development resumes

Phases 0, 1, and 2 are complete and green. After this corrected plan passes the unchanged workflow, create `ui_eh_ph02a_mainactivity_modularization` from the latest green head of `ui_eh_ph02_screen_routing`, not directly from the older code-only commit.

Proceed through the five Phase 2A extraction waves with a green build after each wave. Start Phase 3 only after the complete 95-test Phase 2 regression, added Phase 2A tests, visual verification, and the manual serial safety net are green. In particular, do not build the version-history UI before the v3-to-v4 migration, encrypted snapshot logic, rollback behavior, and retention tests are complete.